

Feature: Service pages

From build-plan: feature 5

Branch: `feature/service-pages`

Goal

`/nails` , `/lashes` , `/makeup` and their `/es` twins: one page per service, each owning its own search terms, each carrying what that service costs and how long it takes.

Three pages and not one, because a single services page competes with itself and ranks for none of them. This is the feature that turns "nails in Calgary" from a phrase in the home page copy into a page that can actually win it.

In scope

- A `Treatment` record type and `data/treatments.ts` , keyed against the dictionary the way `faq.ts` and `gallery.ts` already are.
- Per-service page copy in both dictionaries: page metadata, an H1, a lede, and one label per treatment.
- One shared route, `app/[locale]/[service]/page.tsx` , that renders all three services from `generateStaticParams` .
- A treatment list per service: label, duration, price.
- A recent-work strip per service, reusing `GalleryFigure` and the records already in `data/gallery.ts` .
- `Service` JSON-LD per page, built from the same data the page renders.
- Each page's own metadata, canonical and hreflang.

Out of scope

- **An FAQ on the service pages.** `FaqItem.serviceId` exists and is `null` on all five entries, so the only FAQ available is the site-wide one already on the home page. Repeating those five questions across four pages is the duplicate-content mistake the overview's "no page borrows another's words" rule exists to prevent, and writing fifteen new bilingual questions is content work nobody has done. Leave `serviceId` unused for now.
- **Booking from a service page.** The CTA goes to `/contact` , which is item 7.
- **Sitemap and robots.** Item 9, even though three new routes land here.
- **Changing the nav or footer.** They already link to all three.
- **A lightbox on the work strip.** Same decision as the gallery.

- **Per-service gallery filtering by URL.** Feature 4 settled that the filter is a view control, not a destination.

Build loop

`blueprint/config.json` sets `workflow.stepReview: "feature"` and `workflow.checkpointCommits: "disabled"`: work the steps in order, no commits during implementation, one review at the end. `/complete` makes the single commit.

No declared Verify command. The gate is `npm run build` (TypeScript runs inside it) and `npm run lint`, plus looking at the pages. Baseline on `main` at spec time: build passes, four static routes.

Steps 3 and 4 are gated on content that does not exist. See Open questions. Steps 1, 2, 5 and 6 can run today and turn three 404s into three real pages.

Build steps

- ☒ **1. The `Treatment` record and the per-service copy.** `data/treatments.ts` exporting a `Treatment` type and a `treatments` array, keyed off `Dictionary["services"][id]` `["treatments"]` so a treatment with no label is a compile error. Extend the existing `services.<id>` block in both dictionaries with `meta`, `headingLead`, `headingAccent`, `lede`, `eyebrow` and an empty `treatments` object. Shapes in Data / contracts. **Done when:** `npm run build` passes, and removing one treatment label from `es.ts` fails it.
- ☒ **2. The shared route, page head and CTA.** `app/[locale]/[service]/page.tsx` with `generateStaticParams` over the three service ids crossed with the two locales, `dynamicParams = false`, its own `alternates`, the eyebrow/H1/lede, and the lifted `BookingBand`. No treatment list and no work strip yet. **Done when:** the build route table lists all six service routes as prerendered; `/nails`, `/lashes`, `/makeup` and the three `/es` twins render their own H1 and lede; `/en/nails` 301s to `/nails`; each page's canonical and three hreflang links point at that service, not at `/`; and an unknown slug such as `/facials` 404s.
- ☒ **3. The treatment list.** *Built and proven; no real prices exist yet.* Label, duration and price per row, ordered, with the service accent as a rule rather than as the price colour. **Done when:** every treatment in `data/treatments.ts` renders on its service's page in both locales, prices read as whole dollars and durations in the shared format, and a service with no treatments renders its empty state instead of an empty table.
- ☒ **4. The recent-work strip.** *Nails only; lashes and makeup have no photos.* Up to six of that service's photographs from `data/gallery.ts`, reusing `GalleryFigure`, with a link through to `/gallery`. **Done when:** `/nails` shows its photographs, a service with none renders no strip at all rather than an empty row, and the link to the gallery works in both locales.
- ☒ **5. Service JSON-LD.** `serviceSchema` in `lib/schema.ts`, built from the same records the page renders, following `faqPageSchema` and `imageGallerySchema`. **Done when:** each page emits exactly

one `Service` block naming the service, the provider and Calgary; it omits the offer catalogue entirely when that service has no treatments rather than emitting an empty one; and `npm run build` and `npm run lint` pass clean.

- ☒ **6. The closing pass.** Re-read all six pages against "nothing ships with placeholder text" and against the one-page-one-vocabulary rule. **Done when:** each page's title, H1 and lede carry Calgary and that service's own terms, no two pages share a sentence, and the final build and lint pass.

Files / areas

New:

- `app/[locale]/[service]/page.tsx`
- `components/service/TreatmentList.tsx` - server, the price list
- `components/service/WorkStrip.tsx` - server, wraps `GalleryFigure`
- `data/treatments.ts`
- `lib/format.ts` - the price and duration formatters

Changed:

- `dictionaries/en.ts`, `dictionaries/es.ts` - extend each `services.<id>` block
- `lib/schema.ts` - `serviceSchema`
- `data/services.ts` - only if `Service` needs a `treatments` accessor; prefer a helper in `data/treatments.ts` and leave `services.ts` alone

Unchanged and worth knowing: `lib/locale.ts` already lists `/nails`, `/lashes` and `/makeup` in `routes`, so `next.config.ts`'s rewrites and the `/en` redirect already cover them. The header and footer already link to all three. No routing or config change is needed beyond the page file.

Data / contracts

`Treatment` in `data/treatments.ts`, keyed against the dictionary the way `data/faq.ts` and `data/gallery.ts` are:

```
export type TreatmentKey<S extends ServiceId> =
  keyof Dictionary["services"][S]["treatments"];

export type Treatment = {
  key: string;
  serviceId: ServiceId;
  /** Whole Canadian dollars. Not cents, not a range. */
  priceCad: number;
  /** True when the price is a starting point. Renders as "From $5". */
```

```

    from?: boolean;
    durationMinutes: number;
    order: number;
  };

```

The per-service key relation is the fiddly part: a treatment's `key` must exist under *its own* service's `treatments` block, not under any service's. If the generic above proves awkward in practice, a discriminated union of three per-service record types is the fallback - but the compile-time guarantee that a treatment cannot carry a label from another service must survive, because that is what stops a lash price appearing on the nails page.

Price rendering: \$65 , in both locales. Whole dollars, no cents, a leading dollar sign. Deliberately not `Intl.NumberFormat` with the page locale: Spanish convention puts the symbol after the number (65 \$), which is wrong on a Canadian price list that a bilingual visitor will compare against the English page. The number is the same fact in both languages, and the overview locks price as shared rather than locale-keyed for exactly that reason.

Duration rendering, one shared formatter, no locale strings needed because `h` and `min` are the same abbreviation in both languages:

```

| durationMinutes | Renders | |---|---| | 45 | 45 min | | 60 | 1 h | | 90 | 1 h 30 min | | 150 | 2 h 30 min |

```

Dictionary shape, extending the existing `services.<id>` block rather than starting a second one:

```

services.<id>.name           already exists
services.<id>.blurb          already exists, used by the home cards
services.<id>.meta.title     carries Calgary and this service's terms
services.<id>.meta.description
services.<id>.eyebrow
services.<id>.headingLead / .headingAccent
services.<id>.lede
services.<id>.empty.heading / .body for a service with no treatments yet
services.<id>.treatments.<key> one label per treatment
services.<id>.work.heading / .all the work strip heading and its gallery link

```

Each page owns its vocabulary. From the overview: nails owns gel, acrylic, extensions, chrome and nail art; lashes owns classic, hybrid, volume and fills; makeup owns grad, bridal, event and glam. No page uses another's terms, and the home page's service blurbs are not reused as the page lede - they already rank for the home page.

Metadata per page, set explicitly for the same reason the gallery needs it: the layout points `canonical` and all three `hreflang` values at `/`, and a child's `alternates` replaces rather than merges.

```

alternates: {
  canonical: localePath(locale, service.href),
  languages: {
    en: localePath("en", service.href),
    es: localePath("es", service.href),
    "x-default": localePath(defaultLocale, service.href),
  },
}

```

Route shape. One dynamic `[service]` segment, `generateStaticParams` returning the three ids for each locale, `dynamicParams = false` so an unknown slug 404s rather than rendering an empty page. The slugs come from `siteConfig.nav / services[].href` with the leading slash stripped, so they stay defined in one place. Whether the Spanish slugs ever translate is still open in the overview; this feature keeps them mirrored and changes nothing in `lib/locale.ts`.

Service JSON-LD, via the existing `jsonLd` helper:

```

{ "@context": "@type": "Service", name, description,
  serviceType, areaServed: "Calgary",
  provider: { "@type": "LocalBusiness", name: siteConfig.business.name },
  hasOfferCatalog: { ... } // omitted entirely when there are no treatments
}

```

`provider` reads `siteConfig.business.name`, never a literal. The offer catalogue is omitted rather than emitted empty, the same rule the gallery's `ImageGallery` follows.

Testing

No unit tests: this project has no test runner and `/tests` has not been run. No browser harness either, so nothing here claims automated browser coverage.

The gate is `npm run build` and `npm run lint` plus the manual checks in each step. One destructive check is run once and reverted, not committed: removing a Spanish treatment label to prove the build fails.

Check all six routes, not three. A service page is the easiest thing on this site to finish in English and forget in Spanish.

Notes for the AI

Colour is navigation. Pink is nails, violet lashes, orange makeup, keyed off `service.id` the way `ServiceCards` and `GalleryFigure` already do it. Reuse that pattern rather than writing a fourth accent

map.

The accent fills fail contrast under 18px. Prices and durations are small text. On a light surface they use the `-on-light` twins, never the fill. Do not colour a price with the service accent just because it is available.

No component contains the word Dani. The provider name in the schema, the business name in any copy, all from `siteConfig`.

Reuse, do not re-derive. `Band`, `BandHead`, `Hot` and `BookingBand` are in `components/site/` and were lifted there by feature 4 precisely so this feature could use them. `GalleryFigure` already handles the missing-endpoint placeholder, the hover scrim and the service tag.

Three pages, one template, three vocabularies. The template is shared; the words are not. If a sentence would read correctly on two of the three pages, it is the wrong sentence.

Animation is CSS. Do not add a package.

Open questions

1. There are no prices and no durations. This blocks step 3. `data/` contains no treatment, price or duration anywhere, and the build-plan line for this item is literally "treatments, prices and durations". These are business facts only Dani can supply, and they must not be invented - a wrong price on a price list is worse than no price list.

What is needed, per service:

- which treatments she offers
- what each costs, in whole Canadian dollars
- how long each takes, in minutes

Partial answers are fine and the page handles them: a service with no treatments renders its empty state, exactly as the gallery did with no photographs.

The home FAQ already implies some durations - "a gel manicure is about an hour", "a full set with extensions and art runs two to three hours", "lash sets are around two hours, fills about an hour". Those are prose she approved, not data, and the two must not disagree once the table exists.

2. Resolved: a price needs to be able to be a starting price. Market research on 2026-09-21 settled this. Every real Calgary salon menu read uses open-ended pricing somewhere - nail art "\$5 & up", French or ombré "\$15 & up", extra-long nails - because a full set with art genuinely costs more than the same set plain. So `Treatment` carries a `from` flag:

```
/** True when the price is a starting point rather than the whole charge. */  
from?: boolean;
```

Rendered as `From $5` against a plain `$45` , with the word coming from the dictionary (`services.<id>.priceFrom` , or one shared string) because "From" and "Desde" do translate even though the number does not. Default is `false` : an exact price is the normal case and the flag is the exception.

3. The lashes page will be nearly empty, and that is an SEO risk. Lashes has no photographs at all in the 72 she sent, no lash-specific FAQ, and no prices yet. Its page would be an H1, a lede and a CTA. A thin page competing for "lashes Calgary" can underperform or go unindexed, and it dilutes a new domain's crawl budget.

Three ways, and this is a product decision rather than a code one:

- **Ship all three anyway.** Honest, and the page improves as content lands. The nav already links to it, so today it is a 404 - a thin page beats that.
- **Ship nails and makeup, hold lashes** until there is something on it. Cleanest for SEO, but it contradicts the build plan's "one page each" and leaves a nav link dead.
- **Ship all three but keep lashes out of the sitemap** until it has content. That is item 9's territory, so it would need noting there rather than doing here.

Spec'd as the first option, because a real page with a real lede beats a 404 and because the fix is content she is already being asked for. Change it now if you would rather hold the page back.

Status: verified against build output, not against a rendered page

`npm run build` exit 0, `npm run lint` exit 0. The route table gained `/[locale]/[service]` with all six paths prerendered.

Parsed out of the prerendered HTML with the RSC payload stripped first:

```
|| /nails | /lashes | /makeup | |---|---|---|---| H1 carries Calgary (en) | yes | yes | yes | | Title carries
Calgary | yes | yes | yes | | Canonical | /nails | /lashes | /makeup | | hreflang trio | own route | own
route | own route | | Photographs | 6 of 10 | 0, strip omitted | 0, strip omitted | | Price list | empty state |
empty state | empty state | | Service JSON-LD | yes | yes | yes | | areaServed | Calgary | Calgary |
Calgary | | provider | Dani Moreno, from siteConfig | same | same | | hasOfferCatalog | omitted |
omitted | omitted |
```

The three Spanish twins render the same structure with their own copy. Ledes are distinct across all three pages.

The price list was proved against temporary rows, then reverted. With three nails rows and two lashes rows injected:

Gel manicure	Takes 1 h	\$45
Full set, extensions	Takes 2 h	\$60
Custom art, per nail	Takes 30 min	From \$5

Spanish rendered `Dura` and `Desde`. Makeup kept its empty state while the other two showed rows, so the empty branch and the populated branch were both observed in the same build. `hasOfferCatalog` appeared with three `Offer` entries priced in CAD and disappeared again when the rows were removed.

The missing-translation guard was re-proved at the new nesting depth. Removing the Spanish `fullSet` treatment label failed the build:

```
ictionaries/es.ts(96,7): error TS2741: Property 'fullSet' is missing in type  
'{ gelManicure: string; art: string; }' but required in type  
'{ gelManicure: string; fullSet: string; art: string; }'
```

All temporary rows and labels were removed; `data/treatments.ts` is back to three empty arrays and both dictionaries to `treatments: {}`.

Departures from the spec

1. **The `Treatment` shape changed from a flat list to one array per service.** The spec proposed `{ key, serviceId, ... }[]` with a generic key type and named a discriminated union as the fallback. Neither was needed: keying the whole structure by service - `{ [S in ServiceId]: TreatmentRow<S>[] } -` gets the same guarantee more simply, because the service is the object key rather than a field that could disagree with it. 2. **The label lookup needs one cast, isolated in `treatmentLabel`.** TypeScript cannot index `Dictionary["services"][S]["treatments"]` while `S` is generic; it reports `TS2536 cannot be used to index type '{ } | { } | { }'`. The cast is read-side only. The guarantee the spec insisted must survive does survive, because it lives at the authoring site: a nails row still cannot be written with a lashes key, and a row whose label is missing is still a compile error. 3. **Each service gained its own `cta` block,** which the spec did not plan. The page was first wired to `t.gallery.cta`, which put "Seen one you want?" on a lashes page and the same four sentences on four pages - the exact duplicate-content mistake the ledes were written to avoid. 4. **The makeup H1 was rewritten.** "Makeup for the day that matters" was better brand voice and carried no Calgary, which the overview requires in the H1 of every page. It is now "Grad and event makeup in Calgary", matching the lashes pattern. 5. **All three ledes were rewritten to carry Calgary,** each phrased differently so the pages still share no sentence. 6. **A real space was added before the price when `from` is set.** The gap was CSS margin only, so the text content read `From$5` to a screen reader and to anyone copying the line.

Not verified

- **Nothing was seen rendered in a browser.** No dev server was started. The price list at phone width, the two-column work strip, the focus rings and the heading scale are argued from markup and CSS, not observed.
- **`/facials` returning 404** is argued from `dynamicParams = false` plus a `generateStaticParams` that returns only three slugs. Not observed.

- `/en/nails` redirecting to `/nails` is argued from the `next.config.ts` rule that already works for `/en` and `/en/gallery`. Not observed.
- **No prices are live on any page.** All six routes show "Prices are being set". That is real copy rather than a placeholder, but the build-plan line for this item reads "with treatments, prices and durations", and two of those three are still absent.
- **The Spanish copy is draft.** Dani is a native speaker and has not read it.

Implementation walkthrough

The data shape, and what it is protecting

`data/treatments.ts` is keyed by service rather than being one flat list:

```
export type Treatments = { [S in ServiceId]: TreatmentRow<S>[] };
```

That single decision is what makes the label relation checkable. `TreatmentRow<S>` constrains `key` to the keys of *that* service's `treatments` block in the dictionary, and the service is the object key rather than a field, so there is no way to write a lash price under nails or to reference a label that does not exist. The failure this prevents is not hypothetical - a price list that silently shows the wrong service's label is wrong in a way nobody notices until a client does.

The cost is one cast, in `treatmentLabel`, because TypeScript will not index `Dictionary["services"]
[S]["treatments"]` while `S` is still generic. It is isolated in a named function with a comment explaining precisely what it does and does not weaken, rather than sprinkled at each call site.

One route, three vocabularies

`app/[locale]/[service]/page.tsx` renders all six pages. `generateStaticParams` enumerates only its own segment and Next crosses it with the locales from the layout above; `dynamicParams = false` means anything that is not one of the three slugs is not a page at all.

The slugs come from `serviceSlug()`, which strips the leading slash off the `href` already in `data/services.ts`. That keeps the URL and the link in the header from ever drifting: there is one place a service's path is written.

`[service]` sits beside the static `gallery` segment. Next resolves a static segment first, so `/gallery` is still the gallery and only the three service slugs reach this file.

The template is shared and the words are not. Every string comes from that service's own dictionary block, and the rule in the comment above `services` is the test: if a sentence would read correctly under another service, it is the wrong sentence. That rule caught three real problems during the closing pass -

the makeup H1 with no Calgary, three ledes with no Calgary, and all three pages reusing the gallery's closing CTA.

The price list

`TreatmentList` is a definition list rather than a `<table>`. Three columns suggests a table, but each row is really a label with two facts attached, and a two-column price list collapses on a phone in a way a three-column table does not. The phone is the product.

Its empty state is the only thing any visitor sees today, so it is written as real copy - a new artist still setting her prices - rather than a stub. That also keeps three routes the header already links to from being dead ends, which is what they were before this feature.

`lib/format.ts` holds both formatters. Price is `$65` in both languages, not `Intl.NumberFormat` keyed to the locale: Spanish convention puts the symbol after the number, and `65 $` on a Canadian price list that a bilingual visitor will compare against the English page is wrong. Duration needs no dictionary strings at all, because `h` and `min` are the same abbreviation in both languages - which is the whole reason it lives in one file rather than two dictionary blocks.

The work strip

`WorkStrip` reuses `GalleryFigure` rather than introducing a second figure, so the hover scrim, the service tag and the missing-endpoint placeholder behave identically here and on the gallery.

It returns `null` when a service has no photographs. That is not defensive tidiness: lashes has none, makeup's two are held pending the model's consent, and a heading over an empty row would read as a broken section rather than an absent one. Only `/nails` shows a strip today.

Structured data

`serviceSchema` follows the two helpers already in `lib/schema.ts`: same records, same dictionary, so the markup and the schema cannot disagree. `provider` reads `siteConfig.business.name` rather than a literal, which is the template test holding on a fourth page type.

`hasOfferCatalog` is omitted entirely rather than emitted empty when a service has no priced treatments. An empty catalogue is a claim that she offers nothing, which is worse than saying nothing about her offers - the same rule the gallery's `ImageGallery` follows when there are no photographs.